

Tecnologie del Linguaggio Naturale

Appunti compatti per l'orale — Prof. Daniele Radicioni

1 Semantica Lessicale

La **semantica lessicale** studia il significato delle parole e le relazioni fra loro. Le lingue naturali sono **deformabili**: a differenza di matematica o chimica, dove i simboli hanno un significato fisso, il significato di una parola dipende dal contesto e si estende per polisemia e metafora. Non basta quindi un dizionario di forme: serve studiare la *struttura* del significato.

- **Ontologia**: due letture utili. (1) filosofica – studio dell'essere e delle categorie fondamentali. (2) come **concettualizzazione** – la specifica esplicita di come un agente percepisce e organizza un pezzo di realtà, indipendentemente dal vocabolario usato: situazioni descritte con parole diverse possono condividere la stessa concettualizzazione.
- **Relazioni lessicali**: sinonimia (stesso significato); iponimia/iperonimia (sottoclasse/superclasse); meronimia/olonimia (parte-di/tutto); antonimia (significato opposto). Sono la spina dorsale di WordNet.
- **Il lessico non è una tassonomia pulita**: c'è **sovrapposizione dei sensi** (i quasi-sinonimi, che in un'ontologia sarebbero mutuamente esclusivi), ci sono **buchi lessicali** (concetti senza una parola dedicata, serve una perifrasi), e i **lessici tecnici** sono invece molto più vicini all'ontologia del dominio, perché i termini sono pensati apposta per corrispondere alle categorie.
- **Ontologie fondazionali** (valide trasversalmente in domini diversi): DOLCE, SUMO, CYC.

DOLCE

- **Endurant**: presente per intero in ogni istante della sua esistenza, e può *cambiare* nel tempo. Un oggetto fisico esiste sempre per intero e può assumere proprietà diverse.
- **Perdurant**: si estende nel tempo, è presente solo *parzialmente* in ogni istante (a metà del suo svolgersi è avvenuto solo in parte), e per questo non “cambia” propriamente: la sua collocazione spazio-temporale è già determinata dagli endurant che vi partecipano.
- **Partecipazione**: la relazione che lega i due – un endurant esiste partecipando a un perdurant.
- **Quality**: proprietà misurabile di un individuo (es. il colore), da tenere distinta dal suo *valore* (il particolare rosso di quella rosa).
- **Criteri di identità**: condizioni necessarie da condividere perché *A* sia sottoclasse di *B*.

Esempio: Una mela verde che diventa rossa è un endurant: cambia proprietà nel tempo. Una corsa non “cambia”, semplicemente si estende nel tempo: è un perdurant.

2 Knowledge Representation

Un sistema di conoscenza = **linguaggio di rappresentazione** (sintassi per codificare l'informazione) + **regole/operazioni** per manipolarlo. La sola logica proposizionale/predicativa non basta: non permette di aggregare in un unico blocco le formule relative allo stesso oggetto.

Reti semantiche

Grafo: nodi = concetti, archi = relazioni. Relazione chiave: **isA** (eredità per transitività).

- **Grafo relazionale**: sottoinsieme del calcolo dei predicati (archi = predicati, nodi = termini). Limiti: la congiunzione è solo implicita, disgiunzione/implicazione/quantificazione universale sono difficili da esprimere, e le relazioni restano sempre binarie.

- **Reti proposizionali**: i nodi possono essere intere proposizioni, quindi si può esprimere la **negazione** e, via De Morgan, anche la **disgiunzione**.

Eredità delle proprietà

Le proprietà di un nodo valgono anche per i nodi sottostanti nella gerarchia isA: questo dà **economia di rappresentazione** (non serve ripetere ogni proprietà a ogni livello) e **manutenzione semplice**. Le eccezioni si gestiscono con la **validità per default**: l'eccezione, essendo più vicina al nodo, prevale sul default ereditato dall'alto.

Esempio: “Gli uccelli volano” è il default, ma “il pinguino non vola” è un'eccezione specifica che prevale.

Ereditarietà multipla ($C \text{ isA } A$ e $C \text{ isA } B$): la struttura non è più un albero ma un grafo, la ricerca passa da lineare a esponenziale, e in caso di conflitto **non esiste un criterio generale** per risolverlo – dipende dal tipo di visita, in profondità o in ampiezza.

Nixon Diamond (caso da manuale)

Nixon è quacchero ($\text{isA} \rightarrow \text{pacifista}$) e repubblicano ($\text{isA} \rightarrow \text{non-pacifista}$). Ricerca in profondità: risultato arbitrario, dipende dall'ordine di visita. Ricerca in ampiezza: vince il cammino più corto. Via d'uscita più onesta: considerare la rete **ambigua** (*dissonanza cognitiva*) invece di forzare un'unica risposta.

C'è un problema strutturale in più: le reti semantiche non distinguono **individuo** da **classe** (isA usato sia per appartenenza sia per inclusione), quindi non si distingue una proprietà vera per tutti i membri da una proprietà vera della classe come tale (“gli elefanti sono numerosi” non vuol dire che ogni elefante sia numeroso).

Frame (Minsky, 1975)

Struttura che rappresenta conoscenza generale su situazioni stereotipate, recuperata dalla memoria e adattata al caso specifico. Ha un nome univoco e degli **slot** (con eventuale **valore di default**), riempiti da procedure *if-needed* (calcolano il valore solo quando serve) o *if-added* (scattano quando lo slot viene riempito).

Esempio: Frame del ristorante: ci aspettiamo tavoli, un cameriere, un conto – anche senza esserci mai stati. Se aprendo la porta trovassimo un palombaro, la sorpresa nasce dalla violazione del frame.

- **Script**: struttura di livello più alto che collega intere sequenze di frasi (es. “andare al ristorante”). Senza uno script condiviso, un testo può risultare incomprensibile anche se sintatticamente corretto.
- **Livello di base**: la categorizzazione più naturale per un parlante.
- **Superordinato / subordinato**: generalizzazioni e specializzazioni rispetto al livello di base.
- **Prototipi**: i membri più tipici di una categoria; i nuovi casi vengono giudicati per somiglianza col prototipo.

Esempio: Il passero è un rappresentante migliore di “uccello” rispetto allo struzzo, pur essendo entrambi uccelli a pieno titolo.

Teorie del significato e della categorizzazione

- **Teoria classica** (Aristotele): condizioni necessarie e sufficienti, confini netti, nessun membro più tipico di un altro.
- **Teoria dei prototipi** (Rosch): un concetto è rappresentato da un membro tipico; i nuovi casi si giudicano per somiglianza col prototipo.
- **Teoria degli esempi**: il concetto non è un prototipo astratto ma l'insieme stesso degli esempi concreti già visti.

- **Theory-theory**: i concetti sono incastonati in una teoria ingenua più ampia sul mondo (“uccello” implica già sapere che vola, che fa il nido, che depone le uova).
- **Dual process**: Sistema 1 (implicito, veloce, categorizzazione **non monotona** con eccezioni) vs. Sistema 2 (esplicito, lento, categorizzazione **monotona**, logicamente coerente).

Il significato delle parole

- **Ambiguità contrastativa / omonimia**: sensi scollegati che condividono per caso la stessa forma (la “riva” del fiume e la voce del verbo “arrivare”).
- **Ambiguità complementare / polisemia**: sensi correlati fra loro (“giornale” oggetto fisico vs. istituzione). I verbi sono più polisemici dei nomi.
- **Teoria referenziale**: il significato è la capacità di riferirsi alla realtà (denotazione).
- **Teoria mentalista**: il riferimento è mediato dal concetto nella mente del parlante, quindi si può parlare anche di cose astratte o immaginarie.
- **Teoria strutturale**: il significato è il valore relazionale di una parola rispetto alle altre dello stesso campo semantico.
- **Teoria distribuzionale**: il significato è l’insieme dei contesti in cui la parola compare – è l’idea alla base di VSM e word embeddings.

Metafora geometrica del significato

I significati sono punti in uno spazio multidimensionale: punti vicini = significati simili. È l’intuizione alla base della **cosine similarity** (Cap. 6).

Il significato di una frase nasce dal significato delle parti (**composizionalità**) – ma non sempre: idiomi, metafore e polisemia sono eccezioni. Meccanismi più fini di interazione semantica:

- **Co-composizione**: il complemento specializza il significato del verbo. “Tagliare i capelli” = accorciare, “tagliare il pane” = affettare, “tagliare l’erba” = falciare – stesso verbo, accezioni diverse selezionate dal complemento.
- **Type coercion**: forzatura di tipo semantico.
- **Legamento selettivo**: l’aggettivo seleziona solo una parte del significato del nome. “Computer veloce” riguarda la velocità di calcolo, non le dimensioni fisiche.

3 WordNet, FrameNet, WSD

WordNet (Princeton, 1985): database lessicale online, ispirato alla psicolinguistica della memoria lessicale. Organizza il lessico per *significato*, non alfabeticamente. 4 categorie: nomi (organizzati in vera gerarchia), verbi (relazioni non gerarchiche), aggettivi, avverbi.

- **Matrice lessicale**: colonne = *word form*, righe = *word meaning*. Stessa colonna → **polisemia**; stessa riga → **sinonimia**.
- **Synset**: insieme di forme che esprimono lo stesso significato; se l’elenco dei sinonimi non basta a chiarire il senso, interviene la **gloss** (la definizione testuale).
- **Relazioni semantiche** (fra synset, non fra parole): **sinonimia**; **antonimia** (relazione fra le *forme*, non fra i concetti – “ricco”/“povero” sì, ma “ascesa”/“caduta” no, pur essendo concettualmente opposte); **iponimia** (ako, transitiva asimmetrica); **meronimia** (hasa, transitiva ma più limitata).
- **Nomi**: 25 gerarchie radice (“punti iniziali”), con un **livello di base** che concentra la maggior parte delle caratteristiche. 3 tipi: **attributi** (aggettivi), **parti** (meronimia), **funzioni** (verbi). L’esperimento di Collins&Quillian mostra tempi di reazione più veloci per proprietà dirette (“il canarino canta”) che ereditate (“il canarino vola”, bisogna risalire a “uccello”).
- **Verbi**: pochi ma molto polisemici, senza vera iponimia; al suo posto la **troponimia** (un modo particolare di compiere la stessa azione).

Esempio: “Sussurrare” è un troponimo di “parlare”: è un modo specifico di parlare.

Conceptual similarity

Wu & Palmer

$$cs(s_1, s_2) = \frac{2 \cdot \text{depth}(LCS)}{\text{depth}(s_1) + \text{depth}(s_2)} \quad (\text{LCS} = \text{Lowest Common Subsumer, il primo antenato comune})$$

Shortest Path

$$\text{sim}_{\text{path}}(s_1, s_2) = 2 \cdot \text{depthMax} - \text{len}(s_1, s_2)$$

Leacock & Chodorow

$$\text{sim}_{LC}(s_1, s_2) = -\log \frac{\text{len}(s_1, s_2)}{2 \cdot \text{depthMax}}$$

Da sensi a termini

$$\text{sim}(w_1, w_2) = \max_{c_1 \in \text{sensi}(w_1), c_2 \in \text{sensi}(w_2)} \text{sim}(c_1, c_2) - \text{si prende il massimo su tutte le combinazioni di sensi: i due termini si disambiguano a vicenda.}$$

Word Sense Disambiguation (WSD)

Trovare il senso corretto di una parola polisemica dato il contesto.

- **Lexical sample:** si disambiguano poche parole target selezionate. **All-words:** si disambigua ogni parola del testo.
- **Feature collocazionali:** la posizione delle parole di contesto conta. **Feature bag-of-words:** conta solo presenza/assenza.

Algoritmo di Lesk

Confronta l'insieme di parole del contesto con la *signature* (parole della gloss + esempi) di ciascun senso candidato; vince il senso con overlap massimo.

Esempio: “The bank can guarantee deposits...” – il contesto (*deposits, invests, mortgage*) ha più overlap con la gloss di *bank*₁ (istituto finanziario) che con *bank*₂ (sponda del fiume) → si sceglie *bank*₁.

Corpus Lesk: arricchisce la signature con parole osservate in un corpus annotato, pesate con $\text{IDF} = \log \frac{N_{\text{doc}}}{n_{d_i}}$ invece che con conteggio semplice (le parole funzionali, molto frequenti ovunque, pesano poco).

FrameNet

Collega le parole ai frame che evocano, con evidenze da corpus reali.

- **Unità lessicale (LU):** coppia parola+senso, non solo parola. In WordNet sensi diversi → synset diversi; in FrameNet sensi diversi → spesso frame diversi.
- **Frame Element (FE):** i ruoli semantici propri del frame.

Esempio: Frame REVENGE: FE = *Avenger* (chi si vendica), *Offender* (il colpevole), *Injury* (il danno).

Predicati (evocano il frame, se ne cercano gli argomenti) vs. **filler** (riempiono un ruolo semantico). **Valenza** = numero/tipo di argomenti di un predicato (impersonale, intransitiva, transitiva,

ditransitiva, tritransitiva).

4 NLTK e spaCy

- **Tokenizzazione**: separa il testo in unità (parole/frasi). Primo passo di ogni pipeline.
- **Stopword filtering**: rimuove parole a basso contenuto semantico (articoli, preposizioni) – ma non è mai neutro, va calibrato sul task.
- **Stemming**: riduce alla radice per regole (*helping, helper* → *help*). Errori tipici: **understemming** (parole imparentate restano diverse) e **overstemming** (parole non imparentate diventano uguali).
- **Lemmatizzazione**: riduce alla forma-dizionario (**lemma**), usando il vocabolario reale, non solo regole come lo stemming. Migliora molto se si fornisce il PoS: *leaves* → *leaf* (nome) o *leave* (verbo) a seconda del contesto.
- **PoS tagging**: etichetta ogni parola con la parte del discorso (Penn Treebank: DT, NNP, VBD, ...).
- **NER**: individua frasi nominali che si riferiscono a entità (persone, organizzazioni, luoghi, date...) e ne determina il tipo.

spaCy vs. **NLTK**: più orientato alla produzione. Offre in più **entity linking** (entità → ID in una KB esterna), **similarity**, **text classification**, **matching a regole** (come regex ma su token). La pipeline (`nlp(testo)`) restituisce un Doc con tutte le annotazioni; i componenti si possono **disable** (caricati ma non eseguiti) o **exclude** (non caricati) per velocità.

5 Modelli di Linguaggio e N-grammi

Un **Language Model** assegna una probabilità a una sequenza di parole. Utile per traduzione, correzione ortografica, speech recognition.

Regola a catena

$$P(w_1, \dots, w_N) = \prod_{i=1}^N P(w_i \mid w_{1:i-1})$$

Calcolare esattamente ogni fattore è intrattabile su sequenze lunghe, quindi si usa l'**assunzione di Markov**: la probabilità della parola successiva dipende solo da un numero fisso di parole precedenti, non da tutta la storia.

Assunzione di Markov (bigramma)

$$P(w_i \mid w_{1:i-1}) \approx P(w_i \mid w_{i-1}), \text{ quindi } P(w_{1:n}) \approx \prod_i P(w_i \mid w_{i-1})$$

MLE (stima per conteggio)

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}w_i)}{C(w_{i-1})}$$

Si usano **log-probabilità** (somma invece di prodotto) per evitare underflow numerico: $\log(p_1 \cdot p_2) = \log p_1 + \log p_2$.

Valutazione

Estrinseca: si misura l'effetto sull'applicazione finale. **Intrinseca**: si misura il modello da solo, su un test set separato dal training.

Perplexity

$PPL(LM, W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P_{LM}(w_i | w_{1:i-1}) \right)$ – più bassa = modello migliore (meno “sorpreso” dal test set).

Smoothing

Serve a evitare probabilità zero per bigrammi mai visti in training (altrimenti perplexity infinita).

Laplace / add-one

$$P_{Lap}(w_i) = \frac{c_i + 1}{N + V} \quad (\text{unigrammi}); \quad P_{Lap}(w_i | w_{i-1}) = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + V} \quad (\text{bigrammi, } V=\text{vocabolario})$$

- **Backoff**: se manca l'n-gramma di ordine alto, si retrocede a un ordine inferiore (scelta discreta e un po' brusca).
- **Interpolazione**: combina in modo continuo tutti gli ordini insieme, ed è generalmente preferita al backoff.

Interpolazione

$$P(w_n | w_{n-2}w_{n-1}) = \lambda_1 P(w_n) + \lambda_2 P(w_n | w_{n-1}) + \lambda_3 P(w_n | w_{n-2}w_{n-1}), \text{ con } \sum \lambda_i = 1$$

6 Vector Space Model, Rocchio, LDA

VSM: rappresenta ogni testo come vettore numerico. **Term-document matrix**: righe = termini, colonne = documenti, celle = conteggi. Un documento = vettore di lunghezza $|V|$ (vocabolario), tipicamente molto sparso.

Cosine similarity

$\cos(v, w) = \frac{v \cdot w}{|v||w|} = \frac{\sum_i v_i w_i}{\sqrt{\sum_i v_i^2} \sqrt{\sum_i w_i^2}}$ – normalizza il prodotto scalare per la lunghezza dei vettori (altrimenti vincono sempre i documenti più lunghi). Range $[-1, 1]$, ma $[0, 1]$ con conteggi (mai negativi).

IDF e TF-IDF

$idf_i = \log \frac{N}{n_i}$ (N =doc totali, n_i =doc con il termine i) $\rightarrow w_{i,j} = tf_{i,j} \cdot idf_i$. Premia termini frequenti nel documento e rari nella collezione (le stop-word, presenti ovunque, ottengono $IDF \approx 0$).

Metodo di Rocchio

Si basa sul **centroide**, la media dei vettori di un insieme di documenti:

Centroide

$$\vec{t}_X = \frac{1}{|X|} \sum_{\vec{t}_d \in X} \vec{t}_d$$

Il profilo di una classe combina il centroide dei positivi e quello dei negativi:

Profilo di classe (Rocchio)

$f_k^i = \alpha \frac{1}{|POS_i|} \sum_{d_j \in POS_i} w_{kj} - \beta \frac{1}{|NEG_i|} \sum_{d_j \in NEG_i} w_{kj}$ – premia la vicinanza ai positivi e la distanza dai negativi. Se $\alpha = 1, \beta = 0$: solo il centroide dei positivi.

Linear Discriminant Analysis (LDA)

Assume $x | y=k \sim \mathcal{N}(\mu_k, \Sigma)$: classi con media diversa ma **stessa covarianza**.

Funzione discriminante

$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k$ - si sceglie la classe con δ_k più alto. Nel caso binario è una regola lineare $w^T x + b \gtrless 0$.

Esempio: Perché non basta la differenza fra le medie, come in Rocchio? Se una feature ha varianza alta dentro le classi, è meno affidabile: Σ^{-1} la “pesa di meno” automaticamente, anche se la sua differenza grezza fra le medie sembrava grande.

7 Word Embeddings (Word2Vec)

I vettori VSM sono **lunghi e sparsi** (dim. = $|V|$). Le lingue sono **zipfiane**: poche parole frequenti, moltissime rare $\rightarrow$ problema per le feature sparse (una parola mai vista in training resta ignota al modello, anche se sinonimo di una parola nota).

Ipotesi distribuzionale (Harris/Firth): “a word is characterized by the company it keeps”. Gli **embedding** sono vettori **corti e densi** (dim. 50–1000): funzionano meglio delle feature sparse in quasi ogni task (meno parametri = meno overfitting).

Word2Vec: invece di descrivere il contesto, lo *predice*. Due architetture: **CBOW** (contesto $\rightarrow$ parola, il default di Gensim) e **Skip-gram** (parola $\rightarrow$ contesto, meglio su corpus piccoli/parole rare, sg=1 in Gensim).

Skip-gram with Negative Sampling (SGNS)

1. Parola target + vicino reale = esempio **positivo**.
2. Parole a caso fuori dal contesto = esempi **negativi** (*negative sampling*).
3. Si allena una **regressione logistica** a distinguere i due casi.
4. I pesi appresi = gli embedding finali.

Due matrici: **Embedding** (parola target) e **Context** (parole di contesto); l’embedding finale è spesso la somma delle due.

Costo SGNS

$$C = - \sum_i \left[\sum_{w_j \in ctx(w_i)} \log \sigma(v_{w_j}^o \cdot v_{w_i}^i) + \sum_{w_j \notin ctx(w_i)} \log \sigma(-v_{w_j}^o \cdot v_{w_i}^i) \right]$$
 - sostituisce il softmax (troppo costoso su vocabolari enormi) con due termini sigmoidei: massimizza la vicinanza ai veri vicini, la minimizza rispetto ai negativi campionati.

- **Limiti**: parole *out-of-vocabulary* (nessun embedding se mai viste in training); **meaning conflation problem** (un solo vettore per tutti i sensi di una parola polisemica, es. *bank* finanza+fiume mescolati).
- **FastText**: rappresenta la parola come somma degli embedding dei suoi n-grammi di caratteri $\rightarrow$ gestisce anche parole mai viste.
- **GloVe**: usa statistiche globali di co-occorrenza invece di predizioni locali.

Esempio: Analogie: $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$. Ma occhio ai bias: $\vec{father} - \vec{doctor} \approx \vec{mother} - \vec{nurse}$ riflette stereotipi del corpus, non un fatto semantico neutro.

8 Transformer

Gli algoritmi distribuzionali classici soffrono del **meaning conflation problem**. I **Transformer** imparano **contextualized embedding**: il vettore di una parola cambia in base al contesto della

frase specifica.

- **Architettura** (stack di strati, 2–24 tipicamente): **Encoder** (6 strati $\times$ self-attention multi-head + FFN, con **residual connection** + normalizzazione) e **Decoder** (6 strati $\times$ self-attention *mascherata* + attention encoder-decoder + FFN).
- **Encoder-only** (BERT): attenzione **bidirezionale**, capisce ma non genera bene.
- **Decoder-only** (GPT): attenzione **causale/autoregressiva**, genera token per token.
- **Encoder-decoder** (BART): mapping input $\rightarrow$ output, es. traduzione.
- **FFN**: trasformazione non lineare dopo l'attenzione, applicata posizione per posizione; aiuta ad apprendere feature astratte (ruoli sintattici/semantici).

Self/Multi-head Attention

$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$ – Query (cosa cerco), Key (cosa offro), Value (cosa restituisco se rilevante). 8 teste in parallelo nel Transformer originale, poi concatenate e riproiettate ($d_{\text{model}} = 512$).

Esempio: Analogia panetteria: Query = cosa cerco, Key = nome dei prodotti, Value = il prezzo. Più il prodotto (Key) è vicino a quello che cerco (Query), più peso do al suo Value.

L'attenzione da sola non ha nozione dell'ordine delle parole: serve il **positional encoding**, deterministico e non appreso, sommato all'embedding.

Positional Encoding

$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$, $PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$ – l'embedding finale è $x_t = w_t + p_t$.

Sub-word tokenization (BPE): si parte dai singoli caratteri, si uniscono iterativamente le coppie di simboli più frequenti in nuovi simboli $\rightarrow$ vocabolario compatto, robusto a parole mai viste.

Training

Pre-training (non supervisionato, su testo non etichettato):

- **MLM** (Masked LM): $\sim 15\%$ dei token mascherati, il modello indovina; usa contesto sia a sinistra sia a destra $\rightarrow$ **bidirezionale**.
- **NSP** (Next Sentence Prediction, tipico di BERT): predice se una frase segue l'altra. Introduce il token [CLS] (accumula info da tutta la sequenza via self-attention, usato per classificare) e i **segment embedding**.

Fine-tuning (supervisionato): si riparte dal modello pre-addestrato, si continua con una loss specifica del task (es. cross-entropy per classificazione), input con [CLS]... [SEP].

Limiti: complessità **quadratica** dell'attenzione nella lunghezza della sequenza (problema senza GPU); il positional encoding classico usa posizioni **assolute** (architetture più recenti preferiscono codifiche **relative**).